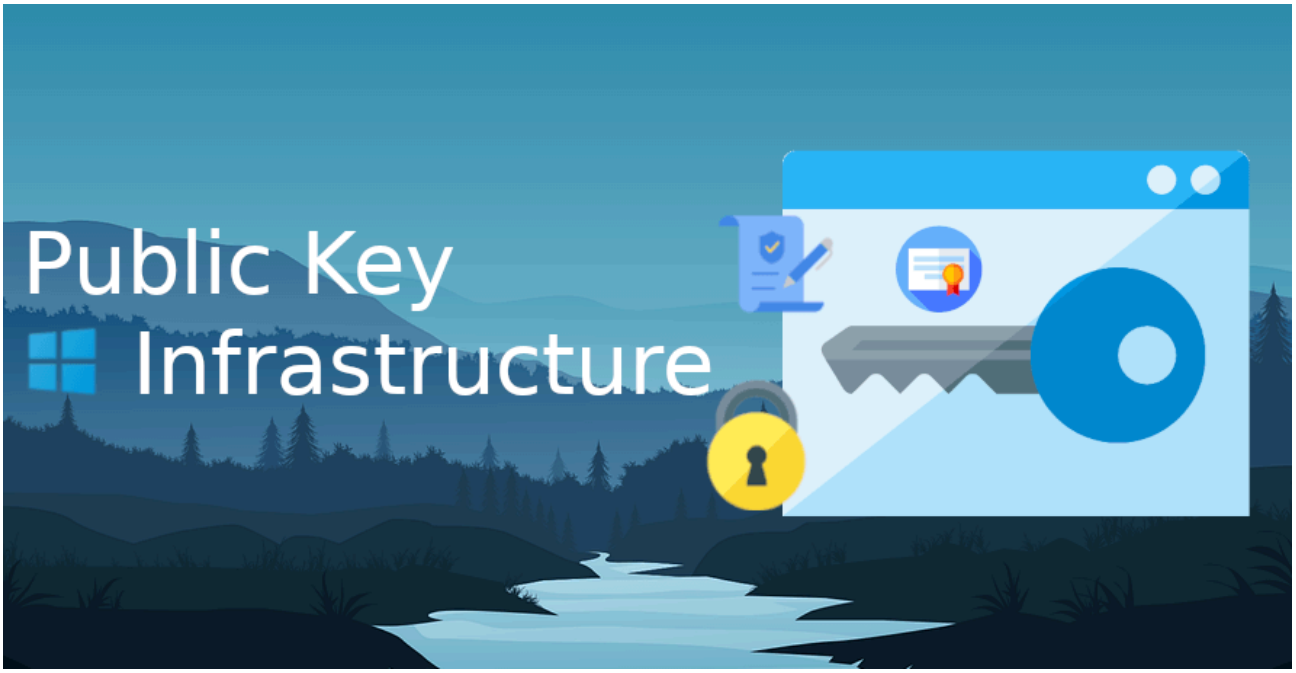
Build a PKI with PowerShell – Part 3 – Offline Root CA – Michael Waterman

michaelwaterman.nl/2026/01/03/how-to-build-a-pki-with-powershell-part-3-offline-root-ca

Michael Waterman January 3, 2026



Other parts in this series

How to: Build a PKI with PowerShell – Part 1 – Preparation

How to: Build a PKI with PowerShell – Part 2 – IIS WebServer

How to: Build a PKI with PowerShell – Part 4 – Enterprise CA

In the previous part, I prepared the PKI Web Server, the semi-public-facing component responsible for distributing CRLs, certificates, and policy information.
In this part, I'll move to the most sensitive and critical component of the entire PKI design: the *Offline Root Certificate Authority*. This system forms the foundation of trust. Everything else in the PKI ultimately depends on it, so it better be very secure!

Why an Offline Root CA?

The Root CA is the trust anchor of your entire PKI. If it is ever compromised, every certificate issued beneath it becomes untrustworthy. For that reason, the Root CA should:

- Be offline by default, meaning not reachable over the network
- Have minimal software installed
- Be used only when absolutely necessary, like key renewal

- Be protected with strong encryption and access controls
- Ultimately be equipped with a High Security Module (HSM) for storing the private key of the CA. It would be a great addition to this blog post but I don't have one laying around to test so it's out of scope for this blog.

Note! Its sole responsibility is to sign subordinate CAs and publish revocation information. It should never issue end-entity certificates.

Step 1 – Preparing the Root CA server

Required permissions

- Local Administrator on the Offline Root CA
- No Active Directory permissions required

Why: The Root CA is standalone and offline by design. All actions, BitLocker configuration, AD CS installation, registry settings, CRL generation, are performed locally. This is a key security principle of the Root CA design.

Before installing any roles, ensure the following:

- The system is *not domain joined*
- All unnecessary services are disabled (host hardening)
- The server has *no permanent network connectivity*
- You have local administrative access

At this stage, the system should be treated as a high-trust asset. From this point forward, every action should be deliberate and documented.

Step 2 – Securing the system with BitLocker

Before installing the Certification Authority role, at a minimum, disk encryption is applied using BitLocker.

Note! *As a Reddit user pointed out, the BitLocker part is applicable to an on-prem situation. If you're following this guide for a Cloud setup, BitLocker is not something you can use in the way I describe. An alternative way of securing your machine would be more appropriate.*

This ensures that:

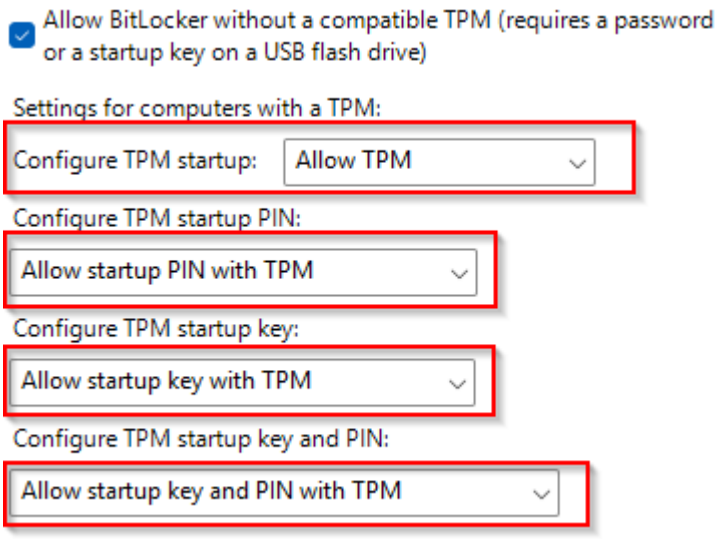
- Private keys are protected at rest
- Offline storage remains secure
- Physical access alone is not sufficient to compromise the system

Before enabling BitLocker on the Offline Root CA, I need to account for an important detail: *this system does not use a TPM*. Because the Root CA is intentionally kept offline and minimal, using a TPM is often impractical or undesirable. Instead, BitLocker will be configured to use a startup password. To allow this, a local Group Policy setting must be adjusted.

```
Computer Configuration -> Administrative Templates -> Windows Components -> BitLocker Drive Encryption -> Operating System Drives -> Require additional authentication at startup
```

Set it to:

- **Enabled**
- *Set all the options to "Allow" as depicted in the picture*. (requires a password or startup key)



This allows BitLocker to function without a TPM, using a pre-boot password instead. Once this policy is applied, BitLocker can be enabled safely using PowerShell. Once enabled, the system may require a reboot before continuing.

Install the BitLocker feature

```
Install-WindowsFeature -Name "BitLocker"
Restart-Computer
```

Enable BitLocker

```
Enable-BitLocker -MountPoint "C:" -EncryptionMethod XtsAes256 -PasswordProtector -Password ( ConvertTo-SecureString -String "P@ssw0rd!" -AsPlainText -Force ) -UsedSpaceOnly -SkipHardwareTest
Add-BitLockerKeyProtector -MountPoint "C:" -RecoveryPasswordProtector
```

Export the Recovery Key

```
(Get-BitLockerVolume -MountPoint "C:").KeyProtector |
Where-Object { $_.KeyProtectorType -eq "RecoveryPassword" } |
Select-Object -ExpandProperty RecoveryPassword |
Out-File "C:\BitLocker-RecoveryKey.txt"
```

> **Tip:** Always treat the BitLocker recovery password as part of your PKI crown jewels. Store it offline, encrypted, and accessible to more than one trusted administrator.

Step 3 – Creating the `capolicy.inf` file

Before I install the Certificate Authority role, I need to create a file called `capolicy.inf` in `C:\Windows`. This file is essentially the CA's instruction manual *during setup* and during key renewal. It allows you to define key PKI defaults ***before*** AD CS generates the CA certificate. Things like:

- Renewal key length
- Renewal validity period
- Hash algorithm choices
- Certain certificate extensions (and whether they should be marked critical)

The important part: ***this file must exist before you install AD CS***. If you create it afterwards, Windows will simply ignore it, well at least for the parts that are valid during the initial setup, more below.

When is `capolicy.inf` used (and why is it "partially" used)?

This is where people often get confused. `capolicy.inf` is read mainly during:

- **Initial CA installation** (when the CA certificate is created)
- **CA certificate renewal** (because "Renewal*" settings apply there)

But it's not a permanent "live config" file. After installation, many CA settings are controlled through registry values and CA configuration commands, which means:

- Some settings are only applied once (installation time)
- Some only matter during renewal
- And some settings are overridden later by explicit configuration steps

So yes, it's powerful, but only at the moment Windows actually looks at it.

If you want a deeper technical breakdown and what each line really does, I wrote a dedicated post about it here: https://michaelwaterman.nl/2025/05/18/pki-part-6-demystifying-the-capolicy-inf-file/

PowerShell: Create the `capolicy.inf` file

This is the exact flow used in your Root CA install script: delete an existing `capolicy.inf` (if present), then build a fresh one with predictable content.

```
$CAPolicy = Join-Path -Path $env:SystemRoot -ChildPath "CAPolicy.inf"

# Remove existing CAPolicy.inf (if it exists) to avoid inheriting old or unintended
settings
if (Test-Path -Path $CAPolicy) {
    Remove-Item -Path $CAPolicy -Force
}

# Create the CAPolicy.inf file (ASCII/ANSI encoding is required for proper
processing by ADCS)
"[Version]" | Out-File -Encoding ascii -FilePath $CAPolicy
Add-Content -Path $CAPolicy -Value 'Signature="$Windows NT$"'
Add-Content -Path $CAPolicy -Value ""

# CA defaults used for installation and future CA certificate renewal behavior
Add-Content -Path $CAPolicy -Value "[Certsrv_Server]"
Add-Content -Path $CAPolicy -Value "RenewalValidityPeriod=Years"
Add-Content -Path $CAPolicy -Value "RenewalValidityPeriodUnits=10"

# Delta CRLs are disabled for an offline Root CA
Add-Content -Path $CAPolicy -Value "CRLDeltaPeriod=Days"
Add-Content -Path $CAPolicy -Value "CRLDeltaPeriodUnits=0"

# Cryptographic defaults
Add-Content -Path $CAPolicy -Value "CNGHashAlgorithm=SHA256"
Add-Content -Path $CAPolicy -Value "AlternateSignatureAlgorithm=0"
Add-Content -Path $CAPolicy -Value ""

# Empty sections prevent CDP and AIA extensions from being added to the Root CA
certificate.
# Actual CRL and AIA publication paths are configured later in the CA
configuration.
Add-Content -Path $CAPolicy -Value "[CRLDistributionPoint]"
Add-Content -Path $CAPolicy -Value "[AuthorityInformationAccess]"
Add-Content -Path $CAPolicy -Value ""

# Force KeyUsage extension to be marked as Critical in the Root CA certificate
Add-Content -Path $CAPolicy -Value "[Extensions]"
Add-Content -Path $CAPolicy -Value "2.5.29.15=AwIBhg=="
Add-Content -Path $CAPolicy -Value "Critical=2.5.29.15"
```

Why I do it this way

- *Delete first*: prevents old config from silently influencing a new CA build.

- *Write in ASCII*: Windows can ignore or misread the file if it's saved with the wrong encoding.
- *Set renewal defaults up front*: these are *hard* to retrofit later without renewal/reinstall.
- *Use placeholders for CDP/AIA*: because I'll configure the real publication URLs explicitly after AD CS is installed.
- For offline Root CAs, it is common practice to omit CDP and AIA extensions from the certificate, as revocation checking is typically not required for the trust anchor.

Note: The "*RenewalKeyLength*" setting under "*[Certsrv_Server]*" is typically not required. By default, the CA will reuse the existing key length during renewal. This setting is only relevant when performing a CA renewal with a new key and explicitly changing the key length. In modern Windows Server versions, it is safe to omit this value unless a key size change is required.

Step 4 – Installing the Certificate Authority role

With the `capolicy.inf` file in place, I can now install the *Active Directory Certificate Services (AD CS)* role. At this stage, I'll only installing the feature itself, not configuring it yet. This ensures that the system is prepared to become a Certification Authority without triggering certificate creation prematurely.

Installing the ADCS role

Run the following command from an elevated PowerShell session:

```
Install-WindowsFeature ADCS-Cert-Authority -IncludeManagementTools
```

This installs:

- The core Certificate Authority role
- Required management tools (if needed)
- Supporting services needed for PKI operations

No certificates are created at this point, that happens in the next step when I explicitly configure the Root CA.

Step 5 – Creating the Root Certification Authority

With the AD CS role installed and `capolicy.inf` in place, I can now create the Root Certificate Authority itself. This is the moment where:

- The root key pair is generated
- The trust anchor for the entire PKI is created
- All future certificates ultimately derive their trust from this instance

Once this step is completed, the identity of your PKI is effectively defined.

Installing the Root CA

The Root CA is installed using the following PowerShell command:

```
Install-AdcsCertificationAuthority  -CACommonName "Corp-Root-CA" `
                                    -CAType StandaloneRootCA `
                                    -CryptoProviderName "RSA#Microsoft Software Key
Storage Provider" `
                                    -DatabaseDirectory
"C:\Windows\System32\CertLog" `
                                    -HashAlgorithmName SHA256 `
                                    -KeyLength 4096 `
                                    -LogDirectory "C:\Windows\System32\CertLog" `
                                    -ValidityPeriod Years `
                                    -ValidityPeriodUnits 10 `
                                    -Confirm:$false
```

What happens during this step

When this command runs, several critical things happen:

- A new private key is generated and stored securely on the system
- The Certification Authority service is installed and initialized
- The CA database and logging structure are created

This is the point of no return, once the Root CA exists, its identity should never change. Important note on immutability, once the Root CA is created:

- The private key ***must never leave the system unprotected***
- The CA name cannot be changed
- Key parameters cannot be modified retroactively

This is why all preparation steps (especially `capolicy.inf`) must be completed before reaching this point.

Step 6 – Configuring CRL (CDP) and AIA publication URLs

Now that the Root CA exists, I need to define where it publishes revocation data (CRL) and where clients can retrieve the Root CA certificate (AIA). This step is important because these URLs are embedded into certificates. If you change them later, you’re basically creating long-term pain: old certificates will keep pointing to old locations. The goal here is simple:

- The Root CA publishes its CRL locally (so I can copy it out).

- Clients retrieve the CRL and Root certificate from the *web server* I built in Part 2 of this series

What I'm doing (high level)

- Remove any default CDP/AIA entries created during setup
- Add a local CRL publish path (CertEnroll folder)
- Add a web-based CDP URL that will be included in issued certificates
- Remove any default AIA entries
- Add a web-based AIA URL pointing to the Root CA certificate

PowerShell code

```
# Define CRL and AIA file templates using ADCS variables
# %3 = CA Name
# %4 = Certificate Name (supports renewal with new key)
# %8 = CRL Name Suffix (supports key renewal)
# %9 = Delta CRL indicator (safe even if delta CRLs are enabled in the future)

$CRLFileTemplate = "<CAName><CRLNameSuffix><DeltaCRLAllowed>.crl"
$AIAFileTemplate = "<CAName><CertificateName>.crt"

$map = @{
    "<CAName>"               = "%3"
    "<CertificateName>"      = "%4"
    "<CRLNameSuffix>"        = "%8"
    "<DeltaCRLAllowed>"      = "%9"
}

# Replace placeholders with ADCS variables
foreach ($k in $map.Keys) {
    $CRLFileTemplate = $CRLFileTemplate.Replace($k, $map[$k])
    $AIAFileTemplate = $AIAFileTemplate.Replace($k, $map[$k])
}

# Remove existing CDP entries to ensure a clean configuration
Get-CACrlDistributionPoint | ForEach-Object {
    Remove-CACrlDistributionPoint $_.Uri -Confirm:$false -Force
}

# Remove existing AIA entries to ensure a clean configuration
Get-CAAuthorityInformationAccess | ForEach-Object {
    Remove-CAAuthorityInformationAccess $_.Uri -Confirm:$false -Force
}

# Local CRL publication path
# CRLs are published to the local CertEnroll directory
# %8 ensures compatibility with CA key renewal
# %9 ensures compatibility if delta CRLs are ever enabled
```

```
Add-CACrlDistributionPoint `
    -Uri "$env:SystemRoot\System32\CertSrv\CertEnroll\$CRLFileTemplate" `
    -PublishToServer `
    -Confirm:$false `
    -Force

# HTTP CDP path included in certificates issued by this Root CA
# This allows relying parties to retrieve the Root CA CRL
Add-CACrlDistributionPoint `
    -Uri "http://trust.domain.suffix/crl/$CRLFileTemplate" `
    -AddToCertificateCdp `
    -Confirm:$false `
    -Force

# Local CA certificate publication path
# %4 ensures support for CA certificate renewal with a new key
Add-CAAuthorityInformationAccess `
    -Uri "$env:SystemRoot\System32\CertSrv\CertEnroll\$AIAFileTemplate" `
    -Confirm:$false `
    -Force

# HTTP AIA path included in certificates issued by the Root CA
# This allows clients to retrieve the Root CA certificate for chain building
Add-CAAuthorityInformationAccess `
    -Uri "http://trust.domain.suffix/crl/$AIAFileTemplate" `
    -AddToCertificateAia `
    -Confirm:$false `
    -Force
```

When working with CDP and AIA paths in ADCS, you'll quickly run into variables like %3, %8, and %9. At first glance they can feel a bit obscure, but they are essential for making your PKI design resilient over time. The table below breaks down what each variable does and, more importantly, which ones actually matter in real-world deployments.

Variable	Name	Description	Practical Use / Notes
%1	ServerDNSName	The DNS name of the CA server	Commonly used in AIA/CDP URLs, but optional
%2	ServerShortName	The NetBIOS name of the CA server	Rarely used
%3	CA Name	The logical name of the Certification Authority	Most important variable – forms the base of file names
%4	CertificateName	The CA certificate name / renewal index	Critical for CA renewal – prevents overwriting CA certificates
%5	Domain DN	Not used in modern ADCS	Legacy, can be ignored
%6	ConfigDN	LDAP path of the forest configuration naming context	Only relevant for LDAP paths
%7	CATruncatedName	Sanitized version of the CA name	Rarely needed
%8	CRLNameSuffix	The CA keypair suffix (e.g., (1))	Essential for CRL handling during key renewal
%9	DeltaCRLAllowed	Indicates whether delta CRLs are used (+)	Prevents conflicts between base and delta CRLs
%10	CDPObjectClass	Object class for CDP entries in AD	Only relevant for LDAP
%11	CAObjectClass	Object class for CA certificates in AD	Only relevant for LDAP

Special note on <CRLNameSuffix> and <CertificateName>

- **<CertificateName>** (mapped to %4) is used to prevent CA certificates from being overwritten during a CA certificate renewal. Each time the CA certificate is renewed, this value is incremented automatically, resulting in filenames such as CA.crt, CA(1).crt, CA(2).crt, and so on.This ensures that previously issued certificates can still build a valid trust chain using the correct version of the CA certificate.

- **<CRLNameSuffix>** (mapped to %8) serves a similar purpose for CRLs, but is only incremented when the CA certificate is renewed *with a new key*.
  In that scenario, the CA must publish separate CRLs for each valid CA key. The suffix allows these CRLs to coexist without overwriting each other, for example `CA.crl`, `CA(1).crl`, `CA(2).crl`.

In short: `<CertificateName>` protects *CA certificate continuity*, while `<CRLNameSuffix>` protects *CRL availability across CA key renewals*.

Why I remove the defaults first

Windows often adds CDP/AIA entries you don't want (local paths, LDAP references, or placeholders). By removing them first, you guarantee the CA publishes only to the endpoints you actually designed for your PKI. I want to be explicit in this setup so I get predictability, not assume the outcome.

Why I publish the CRL locally *and* via HTTP

- **Local publish path (`CertEnroll`)**This makes sure the Root CA generates the CRL in a location where you can easily copy it to the web server later.
- **HTTP CDP URL (`http://trust…/<CAName>.crl`)**This is what clients will use during certificate validation. Since the Root CA is offline most of the time, clients must be able to reach CRLs without contacting the CA.

Why AIA points to the web server

AIA is where clients can fetch the issuing CA certificate. In this case that's the Root CA certificate. By publishing the Root certificate on the web server:

- clients can build trust chains reliably
- the Root CA can remain offline
- certificates stay validatable over time

Small note

Because this is the Offline Root CA, publishing to HTTP doesn't mean the Root CA itself is online. It means I place the files on the web server, and certificates point to that web location. The Root CA remains the vault, the web server is the distribution channel.

Step 7 – Setting CA registry values

At this point the Root CA is installed and the publication URLs are defined, but the CA is *not fully configured yet*. A lot of important CA behavior is controlled through registry-backed

CA settings. Think of this as the “runtime configuration” of AD CS. Some of these values influence:

- how often CRLs are published
- how long certificates issued by this CA are valid
- overlap windows (to prevent validation gaps)
- auditing behavior

When are these settings actually used?

This is the key detail many people miss:

- These values are read by the Certification Authority service (CertSvc) when it starts. Not the settings in the `capolicy.inf` file.
- Some settings only take effect when the CA generates new output, such as:
  - publishing a new CRL
  - issuing a new certificate
  - renewing the CA certificate

So if you change these settings and *don’t restart the service*, you’ll end up in a situation where:

- the registry shows one thing
- the CA continues operating with the old values

That’s why I set the values now and then restart CertSvc before I generate CRLs / export certificates later.

PowerShell: apply the CA registry settings

```
$Certutil = (Join-Path -Path "$env:SystemRoot" -ChildPath "System32\certutil.exe")

Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLPeriod "Years" }
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLPeriodUnits 1 }
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLOverlapPeriodUnits 12 }
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLOverlapPeriod "Hours" }
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\CRLDeltaPeriodUnits 0 }
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\ValidityPeriodUnits (10 / 2) }
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\ValidityPeriod "Years" }
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\AuditFilter 127 }
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\DSConfigDN
"CN=Configuration,DC=domain,DC=suffix" }
Invoke-Command -ScriptBlock { & $Certutil -setreg CA\DSDomainDN
"DC=domain,DC=suffix" }
```

What these settings do (and why you should care)

- **CRLPeriod / CRLPeriodUnits (1 Year)**Defines the validity period of the base CRL. For an Offline Root CA this makes sense: CRLs are published rarely, during controlled maintenance windows.
- **CRLOverlapPeriod / Units (12 Hours)**Adds a safety window so clients don't hit a validation gap if they cache an older CRL slightly longer than expected.
- **CRLDeltaPeriodUnits (0)**Disables Delta CRLs, typical for an Offline Root CA because you're not doing frequent revocation operations here.
- **ValidityPeriod / ValidityPeriodUnits (10 / 2)**
  Sets the default validity for certificates issued by this CA (primarily the subordinate CA certificate). Using *half the Root lifetime* is a common and pragmatic approach: the Root outlives the Issuing CA, but you avoid overly long subordinate validity.
- **AuditFilter (127)**Enables full CA auditing. On a Root CA, this is valuable because every action is high impact and should be traceable.
- **SConfigDN & DSDomainDN**: These settings are especially relevant for offline CAs. It provides the CA with information about the location of the forest's *configuration partition*, which is required when CA certificates or CRLs are published to Active Directory. Even if you do not use Active Directory (LDAP) as an AIA or CDP publication location, it is still considered best practice to store the Root CA certificate and any Policy CA certificates in Active Directory. Doing so allows these certificates to be distributed automatically to domain members, for example through Group Policy or auto-enrollment, without requiring additional manual deployment steps.

Note! These settings can be found in this registry location:
`HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\CertSvc\Configuration\<CA Common Name>\`

Or via:

```
certutil -getreg CA\CRLPublicationURLs
certutil -getreg CA\CACertPublicationURLs
```

Step 8 – Restarting the CA service and publishing the CRL

At this point, all required configuration is in place:

- The Root CA is installed
- `capolicy.inf` has been applied
- CRL and AIA locations are configured
- Registry settings for validity, overlap and auditing are defined

However, none of these settings are active yet. The Certification Authority service only reads these values during startup of the service, so the next step is to restart the service

and trigger the initial publication of the CRL.

Publishing the Certificate Revocation List (CRL)

Now that the service is running with the correct settings, I explicitly publish a new CRL.

This does two important things:

- Generates a fresh CRL using the configured validity and overlap values
- Writes the CRL to the local `CertEnroll` directory

This file will later be copied to the PKI Web Server so clients can retrieve it.

Exporting the Root CA certificate

Next, I export the Root CA certificate itself. This certificate will later be:

- Published via the web server
- Used by clients to build trust
- Installed on the Enterprise CA
- Published to Active Directory

The file will be written to: `C:\Windows\System32\CertSrv\CertEnroll`

Step 9 – Publishing the Root CA certificate and CRL

With the Root CA fully configured and the CRL generated, the final step is to publish the trust material to the web server. This allows clients and subordinate CAs to validate the trust chain without ever contacting the Root CA directly. At this point, the Root CA has done its job. Copy the following files to the CRL directory on the PKI Web Server:

- `C:\Windows\System32\CertSrv\CertEnroll\*.crt`
- `C:\Windows\System32\CertSrv\CertEnroll\*.crl`

To one of these locations:

- `\\trust.domain.suffic\CRL\`
- `C:\inetpub\CDP\CRL`

Once copied, these files will be accessible via HTTP, using the URLs embedded in issued certificates.

Note! Remember that the Root CA is offline, and network connectivity must not be enabled, not even for copying these files. Use an alternate way of secure transfer.

Verifying availability

From another system, with your browser or PowerShell , verify that the files are reachable, e.g.:

```
Invoke-WebRequest -UseBasicParsing -Uri http://trust.domain.suffix/crl/Corp-Root-CA.crt
```

if you see a Status code of 200, your configuration is valid!

And I'm done with this part, so what's next? In the next part, Part 4, I move to the most last component of the entire setup: the *Online Enterprise Certificate Authority*.

See you there!

References

The Microsoft Root Certificate Program

PKI – Part 1: Introduction to Public Key Infrastructure

PKI – Part 2: Choosing the key length and algorithm

PKI – Part 3: The role of hash functions in PKI

PKI – Part 4: Understanding Cryptographic Providers

PKI – Part 5: Creating Unique Object Identifiers (OIDs)

PKI – Part 6: Demystifying the CAPolicy.inf file